

Distributed Join Processing

...

Shimin Luo & Estella Xu

Goal

Implement a lightweight distributed join system in a simulated distributed environment, with a focus on performance optimization.

Implementations:

- Using containerized infrastructure(Docker) and socket-based(TCP) communication
- Using Quest Cluster and MPI

...

Part I: Docker + TCP

Docker Setting

- Each node is encapsulated as an independent container.
- Docker-Compose is used to orchestrate the system, including **one Coordinator node and two Worker nodes**.
- All nodes operate within a shared virtual network with **static IPs** to ensure predictable communication.

```
services:
  coordinator:
    build: ./coordinator
    container_name: coordinator-v1
    hostname: coordinator
    networks:
      mynet:
        ipv4_address: 172.28.1
    ports:
      - "8000:8000"
    volumes:
      # - ./data:/app/data
      # - ./data:/app/data
      # - ./data/shared_data:/a
    environment:
      - NODE_ID=coordinator
      - NODE_TYPE=coordinator
      - MY_IP=172.28.1.10
      - WORKER1_IP=172.28.1.11
      - WORKER2_IP=172.28.1.12
      - WORKER_PORT=9000
      - MY_PORT=8000
  worker1:
    build: ./worker
    container_name: worker1-v1
    hostname: worker1
    networks:
      mynet:
        ipv4_address: 172.28.1.11
    volumes:
      - ./data/worker1:/app/data
    depends_on:
      - coordinator
    environment:
      - WORKER_ID=worker1
      - NODE_TYPE=worker
      - COORDINATOR_URL=http://coordinator:8000
      - COR_IP=172.28.1.10
      - COR_PORT=8000
      - MY_IP=172.28.1.11
      - Other_Worker_IP=172.28.1.12
      - MY_PORT=9000
  worker2:
    build: ./worker
    container_name: worker2-v1
    hostname: worker2
    networks:
      mynet:
        ipv4_address: 172.28.1.12
    volumes:
```

Communication Mechanism

- Nodes communicate using traditional **TCP** socket, without relying on middleware such as RPC (Remote Procedure Call).
- The **Coordinator node** is responsible for **distributing data and SQL** to the workers, also **collecting results via sockets**.
- Each node acts as **both server and client** to enable P2P communication.
- **Concurrent multithreading**: one thread continuously listens for incoming connections, accepts them, and creates new threads to handle each connection.

```
def start_server(port): 1 usage 1 FfffeIXIN *
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind(("0.0.0.0", port))
    s.listen()
    print(f"[Coordinator] Socket server listening on port {port}", flush=True)
    return s

> def read_parquet_to_bytes(file_path):...

> def send_bytes(port, data, dest_ip):...

> def read_byte_data(conn, addr):...

def receive_bytes(server_socket): 1 usage 1 FfffeIXIN *
    """
    Continuously accept connections and delegate each to a separate thread for data reading
    """
    try:
        while True:
            conn, addr = server_socket.accept()
            print(f"[Coordinator] Accepted connection from: {addr}", flush=True)
            client_thread = threading.Thread(target=read_byte_data, args=(conn, addr))
            client_thread.start()
    except Exception as e:
        print(f"[Coordinator] Error in receive_bytes: {e}", flush=True)
```

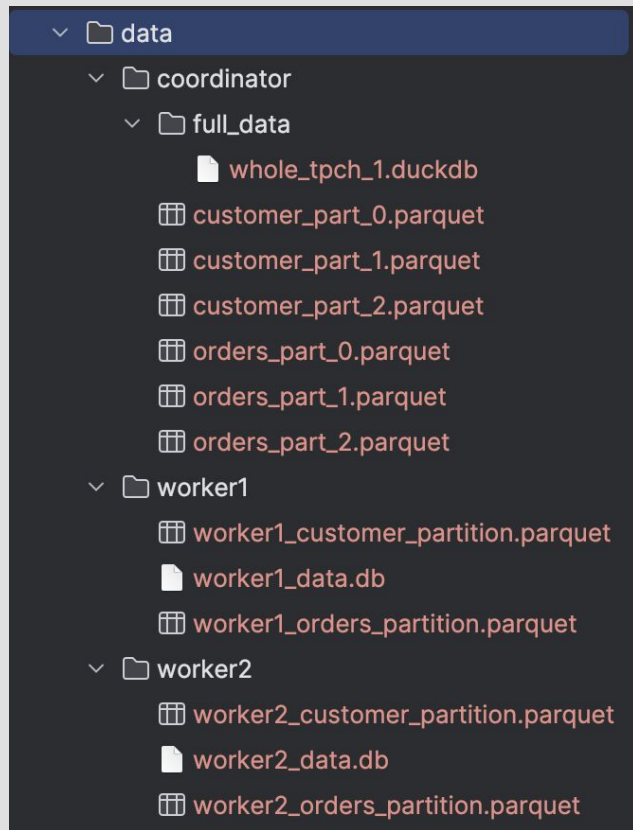
Data Partitioning and Management

- The whole dataset (TPC-H) is loaded by the **Coordinator** and partitioned based on a specified column using **range partitioning**.
- Each partition is **transmitted to one node via socket**, coordinator itself also manages one partition.
- Coordinator maintains a **catalog** that records the **partitioning column for the tables** and the **value ranges** stored at each node.

```
catalog = {  
  "customer": {  
    "total rows": 150000,  
    "partition_column": "c_custkey",  
    "partitions": {  
      "coordinator": [0, 49999],  
      "worker1": [50000, 99999],  
      "worker2": [100000, 149999],  
    }  
  },  
  "orders": {  
    "total rows": 1500000,  
    "partition_column": "o_custkey",  
    "partitions": {  
      "coordinator": [0, 49999],  
      "worker1": [50000, 99999],  
      "worker2": [100000, 149999],  
    }  
  }  
}
```

Data Partitioning and Management

- Each node: **own dedicated directory**, **access permissions** restricted to its own folder.
- The system uses the **Parquet** compressed format, to reduce storage size and, more importantly, to **minimize data transmission volume**.
- Once a worker node gets the signal that data transfer is done, it collects its all Parquet files and builds up its own local database.



Underlying Database Engine — DuckDB

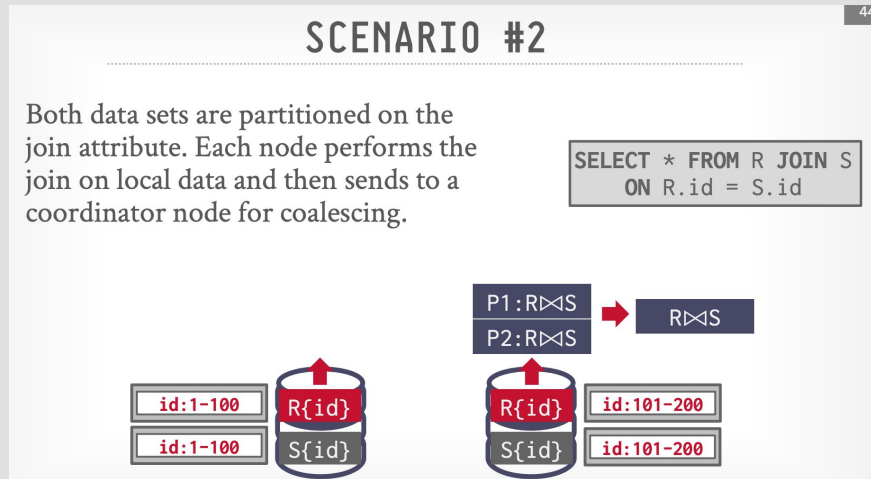
- Each node uses an embedded **DuckDB** instance for local join, and Pandas for data processing.
- We're not concerned with the specific physical operators DuckDB uses to handle local joins, our focus is on implementing the logic for distributed interaction.

```
os.makedirs(name: 'data/coordinator/full_data', exist_ok=True)
con = duckdb.connect(COORDINATOR_FULL_DB)
print(f"[Coordinator] Created full DuckDB at {COORDINATOR_FULL_DB}", flush=True)

con.execute("INSTALL tpch")
con.execute("LOAD tpch")
con.execute("CALL dbgen(sf=1)")
print("[Coordinator] TPC-H data generated (sf=1).", flush=True)
con.close()
```

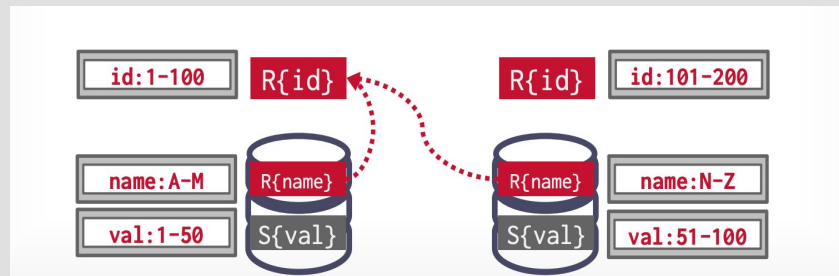
Distributed Join Execution and Scheduling

- Each node initializes its local database.
- When the Coordinator receives a SQL query, it parses the join condition to **extract the join key**.
 - If the join key **matches** the current partitioning key and the partition **ranges align** across workers, the Coordinator directly tells all Workers to **perform a local join in parallel**.
 - If the join key differs from the current partitioning key, a repartitioning is triggered. Then Coordinator and all Workers execute the query locally and concurrently.
- Results ultimately gathered and combined by the coordinator.



Distributed Join Execution and Scheduling

- Each node initializes its local database.
- When the Coordinator receives a SQL query, it parses the join condition to **extract the join key**.
 - If the join key **matches** the current partitioning **key** and the partition **ranges** align across workers, the Coordinator directly tells all Workers to perform a local join in parallel.
 - If the join key **differs** from the current partitioning **key**, a **repartitioning** is triggered. After this phase, Coordinator and all Workers execute the query **locally and concurrently**.
- Results ultimately gathered and combined by the coordinator.



```
38 def extract_join_key(sql: str) -> str:
39     sql = sql.lower()
40     if "join" in sql and " on " in sql:
41         after_on = sql.split(" on ")[1]
42         cond = after_on.split()[0] # e.g., "c.c_custkey=o.o_custkey"
43         if "=" in cond:
44             left, right = cond.split("=")
45             return left.strip().split(".")[1]
46     return None
47
48
49 def check_partition_match(table: str, join_key: str) -> bool:
50     return catalog.get(table, {}).get("partition_column") == join_key
51
52
53 > def repartition_and_send(table: str, new_key: str):
54
55
```

Correctness Verification

- Run the query on single node
- Before comparison, sort and reset the index for both outputs, making sure the column order is consistent.

```
worker1-v1 | c.c_custkey = o.o_custkey
worker1-v1 | WHERE
worker1-v1 | o.o_orderdate < DATE '1995-03-15'
worker1-v1 | ORDER BY
worker1-v1 | o.o_orderdate DESC;
worker2-v1 | customer c
worker1-v1 |
worker2-v1 | JOIN
worker2-v1 | orders o
worker2-v1 | ON
worker2-v1 | c.c_custkey = o.o_custkey
worker2-v1 | WHERE
worker2-v1 | o.o_orderdate < DATE '1995-03-15'
worker2-v1 | ORDER BY
worker2-v1 | o.o_orderdate DESC;
worker2-v1 |
coordinator-v1 | [Coordinator] Local partition0 result written to data/coordinator/local_partition0_result.parquet
coordinator-v1 | [Coordinator] Waiting for Worker results ...
worker2-v1 | [Worker worker2] Query executed in 0.46 seconds
worker2-v1 | [Worker worker2] Query result written to data/worker2_query_result.parquet
worker2-v1 | [Worker worker2] Reading data/worker2_query_result.parquet to bytes.
worker2-v1 | [Worker worker2] Sending result back to Coordinator coordinator:8000
coordinator-v1 | [Coordinator] Accepted connection from: ('172.28.1.12', 58456)
worker1-v1 | [Worker worker1] Query executed in 0.74 seconds
worker1-v1 | [Worker worker1] Query result written to data/worker1_query_result.parquet
worker1-v1 | [Worker worker1] Reading data/worker1_query_result.parquet to bytes.
worker1-v1 | [Worker worker1] Sending result back to Coordinator coordinator:8000
coordinator-v1 | [Coordinator] Accepted connection from: ('172.28.1.11', 46522)
coordinator-v1 | [Coordinator] Received complete Parquet data from Worker
coordinator-v1 | [Coordinator] Writing received bytes to data/coordinator/received_parquet_worker2.parquet
coordinator-v1 | [Coordinator] Received complete Parquet data from Worker
coordinator-v1 | [Coordinator] Writing received bytes to data/coordinator/received_parquet_worker1.parquet
coordinator-v1 | [Coordinator] Loading worker1 result ...
coordinator-v1 | [Coordinator] Loading worker2 result ...
coordinator-v1 | [Coordinator] Concatenating all partitions ...
coordinator-v1 | [Coordinator] Final merged result saved to data/coordinator/final_output.parquet
coordinator-v1 | [Coordinator] Elapsed time: 17.913533 seconds
coordinator-v1 | [Coordinator] Verifying final output against full_data execution ...
coordinator-v1 | [Coordinator] Original query executed in 1.025804 seconds
coordinator-v1 | [Coordinator ✓] Verification passed: distributed join result is correct!
coordinator-v1 exited with code 0
```

Preliminary Experimental Results

- Dataset: TPC-H (SF = 0.1, 0.5, 1, 5)
- query = "SELECT c.c_custkey, c.c_name, o.o_orderkey, o.o_orderdate, o.o_totalprice
FROM customer c JOIN orders o ON c.c_custkey = o.o_custkey
WHERE o.o_orderdate < DATE '1995-03-15'
ORDER BY o.o_orderdate DESC;"
- Docker resource: Memory 7.654GB, Network: 10000Mb/s, I/O: 2.4 GB/s

	SF = 0.1	SF = 0.5	SF = 1	SF = 5
Local join	0.01	0.04	0.09	0.72
Distributed join	0.193192	0.43909	0.787724	17.91353
Join on Single node	0.03169	0.07697	0.10595	1.026

...

Part II: Quest Cluster and MPI

Outline

- Intro to Quest cluster and MPI
- Initial implementation
- Optimizations
 - Collection phase
 - Partitioning phase
- Takeaways

Outline

- **Intro to Quest cluster and MPI**
- Initial implementation
- Optimizations
 - Collection phase
 - Partitioning phase
- Takeaways

HPC Clusters

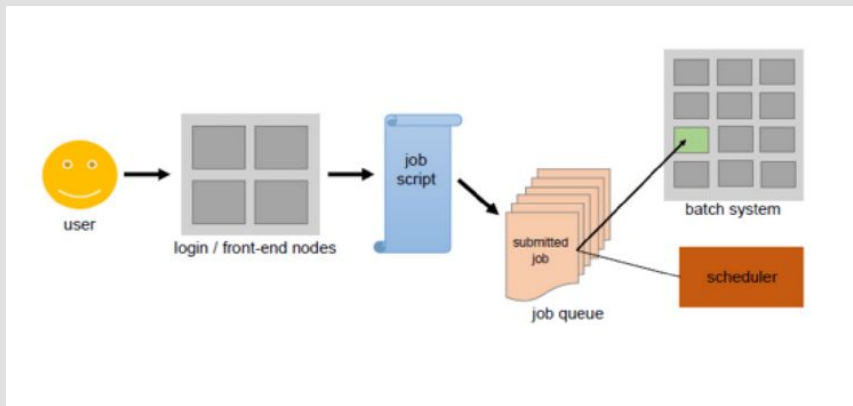
A tightly-coupled set of computers.

- Batch-oriented so the system is shared among multiple users.
- Consists of:
 - Head node (job submission and cluster management)
 - 1 or more compute nodes (execution)

Slurm (Simple Linux Utility for Resource Management) is a software for managing HPC clusters.

Workflow:

1. User submits a request.
2. Head node adds it to job queue.
3. Head node assigns job to a compute node.



Northwestern's Quest Cluster

Submit jobs via bash script. Key parts:

- Number of nodes (includes coordinator node).
- Number of tasks per node (useful if you want to parallelize within a process).
- Output file.
- `mpirun` for running a Python script that uses MPI to access and communicate between at least 1 node.

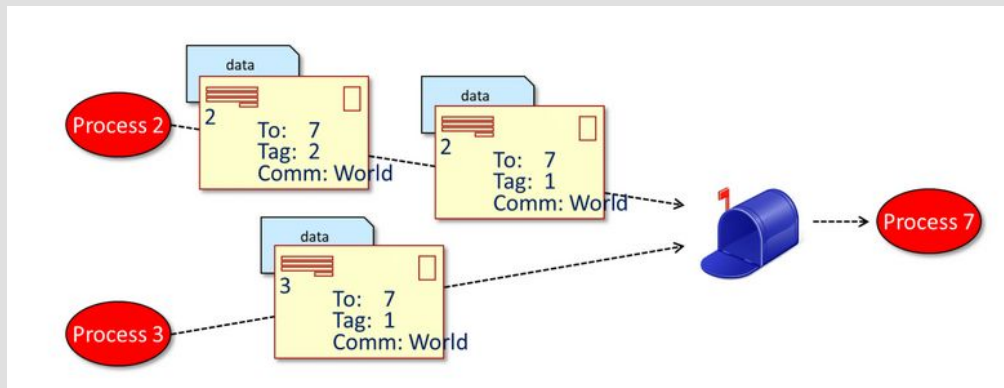
```
#!/bin/bash
#SBATCH --job-name=distributed_join_customer_orders
#SBATCH --account=<account>
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=4G
#SBATCH --time=00:20:00
#SBATCH --partition=short
#SBATCH --output=join_1_2nodes_%j.out
#SBATCH --error=join_1_2nodes_%j.err
```

```
echo "Starting distributed join for customer and orders tables"
echo "Job ID: $SLURM_JOB_ID"
echo "Running on $SLURM_NNODES nodes with $SLURM_NTASKS total tasks"
mpirun -np $SLURM_NTASKS python -u src/distributed_join_customer_orders.py
echo "job_id=$SLURM_JOB_ID"
echo "Distributed join for customer and orders tables finished"
```

MPI

MPI (Message-Passing Interface) offers efficient communication between machines. Consists of:

- Processes (run on each node): independent units of execution (no shared memory).
- Communicator (MPI_COMM_WORLD): allows processes to communicate.
- Rank: unique ID for each process in communicator (rank 0 is typically the coordinator).
- Size: total number of processes.
- Buffer (for each process): temporary storage that holds messages during send/receive.



Blocking vs. Non-Blocking

A **blocking** operation cannot move on until it finishes.

- Returns only when the communication is finished.
- You cannot perform computation while sending.

Non-blocking operations continue execution without waiting.

- Returns immediately.
- Improve performance by overlapping computation and communication.
- To use the data in the buffer, must wait until the communication is finished. But in the meantime, you can do other computation.

MPI Basic Functions

Function Name	Description	Type
send & recv	Performs a blocking send and corresponding receive. → Will not return until the matching receive is invoked.	Point-to-Point
isend & irecv	Performs a non-blocking send and corresponding receive. → Must then use MPI_wait() to use data in buffer.	Point-to-Point
barrier	Forces processes to wait for all to finish (a synchronization point).	Collective
gather	Collect data from many to one.	Collective

Outline

- Intro to Quest cluster and MPI
- **Initial implementation**
- Optimizations
 - Collection phase
 - Partitioning phase
- Takeaways

3 Phases

1. Partition and distribute data
2. Perform local joins with DuckDB
3. Collect results from nodes and make final result

Main Function

```
def main():
    # Initialize MPI
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    total_start_time = datetime.now()

    if rank == 0:
        print("=" * 60)
        print(f"Distributed join customer and orders tables on {size} nodes")
        print("=" * 60)

    # partition the customer table and the orders table
    partition_start_time = datetime.now()
    conn = partition_and_distribute_customer_and_orders(comm, rank, size)

    # synchronize all processes after partitioning and distributing
    comm.Barrier()
    partition_time = datetime.now() - partition_start_time

    # perform local join
    local_join_start_time = datetime.now()
    local_results = perform_local_join(rank, conn)
    local_join_time = datetime.now() - local_join_start_time
```

```
# collect results
collection_start_time = datetime.now()
collect_results(comm, rank, size, local_results)
collection_time = datetime.now() - collection_start_time

# cleanup
cleanup(rank)

total_time = datetime.now() - total_start_time
if rank == 0:
    print(f"total_time: {total_time.total_seconds()} seconds")
    print(f"partition_time: {partition_time.total_seconds()} seconds")
    print(f"local_join_time: {local_join_time.total_seconds()} seconds")
    print(f"collection_time: {collection_time.total_seconds()} seconds")
```

Benchmarks

For each number of nodes (1, 2, 4, and 8), we measure

- Total Time: nodes are initialized → the join is complete.
- Partition Time: depends on...
 - Method of sending files to worker nodes
 - All files must be synchronized (use MPI Barrier() function)
- Local Join Time: depends on...
 - DuckDB performance
 - And thus, how large the local tables are
 - More nodes = smaller tables for each node = faster
- Collection Time: depends on...
 - Method of sending local join results

Partition Code

The coordinator node:

1. Creates a Parquet file for each node containing the sub-table.
2. Sends the Parquet files to the correct node using MPI send.

Each worker node:

1. Receives the Parquet file from node 0 (coordinator).
2. Writes to a local Parquet file.
3. Adds the tables from Parquet to DuckDB.

```
def partition_and_distribute_customer_and_orders_parquet(comm, rank, size):

    db_path = 'data/coordinator/full_data/whole_tpch_0.1.duckdb'
    local_db_path = f'/tmp/node_{rank}.duckdb'
    local_parquet_path_c = '/tmp/customer_local.parquet'
    local_parquet_path_o = '/tmp/orders_local.parquet'

    if rank == 0:

        con = duckdb.connect(db_path, read_only=True)
        for i in range(size):
            part_path_c = f'/tmp/customer_part_{i}.parquet'
            part_path_o = f'/tmp/orders_part_{i}.parquet'

            con.execute(f"""COPY (SELECT * FROM customer WHERE c_custkey % {size} = {i})
                        TO '{part_path_c}' (FORMAT 'parquet')""")
            con.execute(f"""COPY (SELECT * FROM orders WHERE o_custkey % {size} = {i})
                        TO '{part_path_o}' (FORMAT 'parquet')""")

            if i != 0: # coordinator doesn't need to send to itself
                # send both Parquet files
                with open(part_path_c, 'rb') as f:
                    comm.send(f.read(), dest=i, tag=100)
                with open(part_path_o, 'rb') as f:
                    comm.send(f.read(), dest=i, tag=101)
            con.close()

        else:
            file_bytes_c = comm.recv(source=0, tag=100)
            file_bytes_o = comm.recv(source=0, tag=101)

            with open(local_parquet_path_c, 'wb') as f:
                f.write(file_bytes_c)
            with open(local_parquet_path_o, 'wb') as f:
                f.write(file_bytes_o)

    if rank == 0:
        # coordinator reads its own local partitions (you don't send and recv to yourself)
        local_parquet_path_c = f'/tmp/customer_part_{rank}.parquet'
        local_parquet_path_o = f'/tmp/orders_part_{rank}.parquet'

    con = duckdb.connect(local_db_path)
    con.execute("CREATE TABLE IF NOT EXISTS customer AS SELECT * FROM read_parquet(?)", (local_parquet_path_c,))
    con.execute("CREATE TABLE IF NOT EXISTS orders AS SELECT * FROM read_parquet(?)", (local_parquet_path_o,))

    return con
```

Collection Code

Each node runs a local join (result is a list of tuples).

Then, the coordinator node (rank 0):

1. Gathers results from all nodes.
2. Merges all the local results.
3. Writes all merged results into the join_result table (line by line).
4. Outputs join_result table to final Parquet file.

```
def perform_local_join(rank, conn):
    start_time = datetime.now()
    query = "SELECT * FROM customer c JOIN orders o ON c.c_custkey = o.o_custkey"
    results = conn.execute(query).fetchall()
    elapsed_time = datetime.now() - start_time

    return results
```

```
def collect_results(comm, rank, size, local_results):
    all_results = comm.gather(local_results, root=0)

    if rank == 0:
        merged_results = []
        for result in all_results:
            merged_results.extend(result)

        # create DuckDB connection
        con = duckdb.connect()

        # create a table from the result list
        con.execute("""
            CREATE TABLE join_result (
                c_custkey INTEGER,
                c_name TEXT,
                o_orderkey INTEGER,
                o_orderdate DATE,
                o_totalprice DOUBLE
            )
        """)

        con.executemany("""
            INSERT INTO join_result VALUES (?, ?, ?, ?, ?)
        """, merged_results)

        # write to Parquet file
        output_path = 'data/joined_results.parquet'
        con.execute(f"""
            COPY join_result TO '{output_path}' (FORMAT 'parquet')
        """)

        con.close()
```

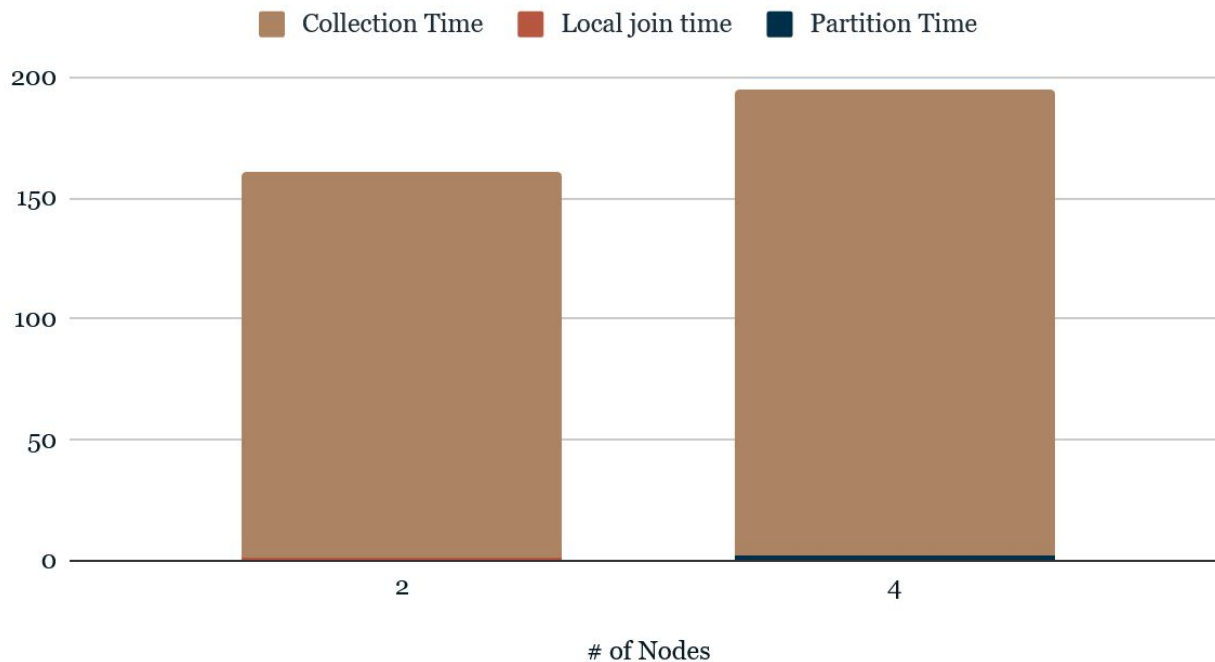
Results (Initial Method)

All times are in seconds.

# of Nodes	Total Time	Partition Time	Local Join Time	Collection Time
2	160.566931	0.70103	0.163714	159.701577
4	194.60694	1.980058	0.099505	192.52769

Results (Initial Method)

Query 1 Join Time: Basic Parquet



Why is this collection so slow?

Step	Description	Why is it slow?
1.	Gathers results from all nodes.	<code>gather</code> is a collective operation that blocks: the coordinator waits for all other processes before proceeding (collective synchronization).
2.	Merges all the local results.	The coordinator takes each of the local result lists and adds to the merged list.
3.	Writes all merged results into the <code>join_result</code> table.	Writing to the table row by row (<code>executemany</code>) is much slower than bulk-loading.
4.	Outputs <code>join_result</code> table to final Parquet file.	N/A

Why is this collection so slow?

Step	Description	Why is it slow?
2.	Merges all the local results.	The coordinator takes each of the local result lists and adds to the merged list.
3.	Writes all merged results into the join_result table.	Writing to the table row by row (<code>executemany</code>) is much slower than bulk-loading.

Comparison for # of Nodes

- Partition time and collection time increase as the number of nodes increases.
 - More work to send files from each node.
- Local join time decreases as the number of nodes increases.
 - Tables sent to each node are smaller when there are more nodes.

# of Nodes	Total Time	Partition Time	Local Join Time	Collection Time
2	160.566931	0.70103	0.163714	159.701577
4	194.60694	1.980058	0.099505	192.52769

Outline

- Intro to Quest cluster and MPI
- Initial implementation
- **Optimizations**
 - **Collection phase**
 - Partitioning phase
- Takeaways

How can we improve the collection phase?

Step	Description	Why is it slow?
2.	Merges all the local results.	The coordinator takes each of the local result lists and adds to the merged list.
3.	Writes all merged results into the join_result table.	Writing to the table row by row (<code>executemany</code>) is much slower than bulk-loading.

For step 2: once a local join occurs, send the resulting table in a Parquet file to the coordinator.

For step 3: bulk load the local tables into the final result table rather than row-by-row.

Collection Code

Each worker node:

1. Runs a local join and puts results into a Parquet file in bulk.
2. Sends Parquet file to coordinator (node 0).

Then, the coordinator node (rank 0):

1. Receives Parquet file from each worker via `recv`. Saves as temporary Parquet file.
2. Inserts into `joined_result` table.
3. Outputs `joined_result` table to final Parquet file.

```
def collect_results_parquet_streaming_1(comm, rank, size, conn):  
  
    collection_time = 0  
    # each node writes local join results to a temporary Parquet file  
    temp_parquet = f'/tmp/join_results_rank_{rank}.parquet'  
  
    query = f"""  
> COPY (...  
    ) TO '{temp_parquet}' (FORMAT 'parquet')  
    """  
  
    start_time = datetime.now()  
    conn.execute(query)  
    local_join_time = datetime.now() - start_time  
  
    if rank != 0:  
        # send parquet file to coordinator with MPI send  
        with open(temp_parquet, 'rb') as f:  
            file_bytes = f.read()  
            comm.send(file_bytes, dest=0, tag=300)
```

Collection Code

Each worker node:

1. Runs a local join and puts results into a Parquet file in bulk.
2. Sends Parquet file to coordinator (node 0).

Then, the coordinator node (rank 0):

1. Receives Parquet file from each worker via `recv`. Saves as temporary Parquet file.
2. Inserts into `joined_result` table.
3. Outputs `joined_result` table to final Parquet file.

```
else:
    start_time = datetime.now()

    # final db for complete join results
    final_conn = duckdb.connect()

    # read coordinator's own local join file first
    final_conn.execute(f"""
        CREATE TABLE join_result AS
        SELECT * FROM read_parquet('{temp_parquet}')
    """)

    # collect parquet files from other nodes using recv for each source rank
    for source_rank in range(1, size):

        # receive file bytes
        file_bytes = comm.recv(source=source_rank, tag=300)

        # write the bytes to a temporary parquet file
        remote_parquet = f'/tmp/join_results_from_rank_{source_rank}.parquet'
        with open(remote_parquet, 'wb') as f:
            f.write(file_bytes)

        # append to main table using duckdb parquet
        final_conn.execute(f"""
            INSERT INTO join_result
            SELECT * FROM read_parquet('{remote_parquet}')
        """)

        # clean up temporary file
        os.remove(remote_parquet)

    # export final result
    output_path = 'data/joined_results.parquet'
    final_conn.execute(f"""
        COPY join_result TO '{output_path}' (FORMAT 'parquet')
    """)

    collection_time = datetime.now() - start_time

    final_conn.close()

# clean up
if os.path.exists(temp_parquet):
    os.remove(temp_parquet)

return local_join_time, collection_time
```

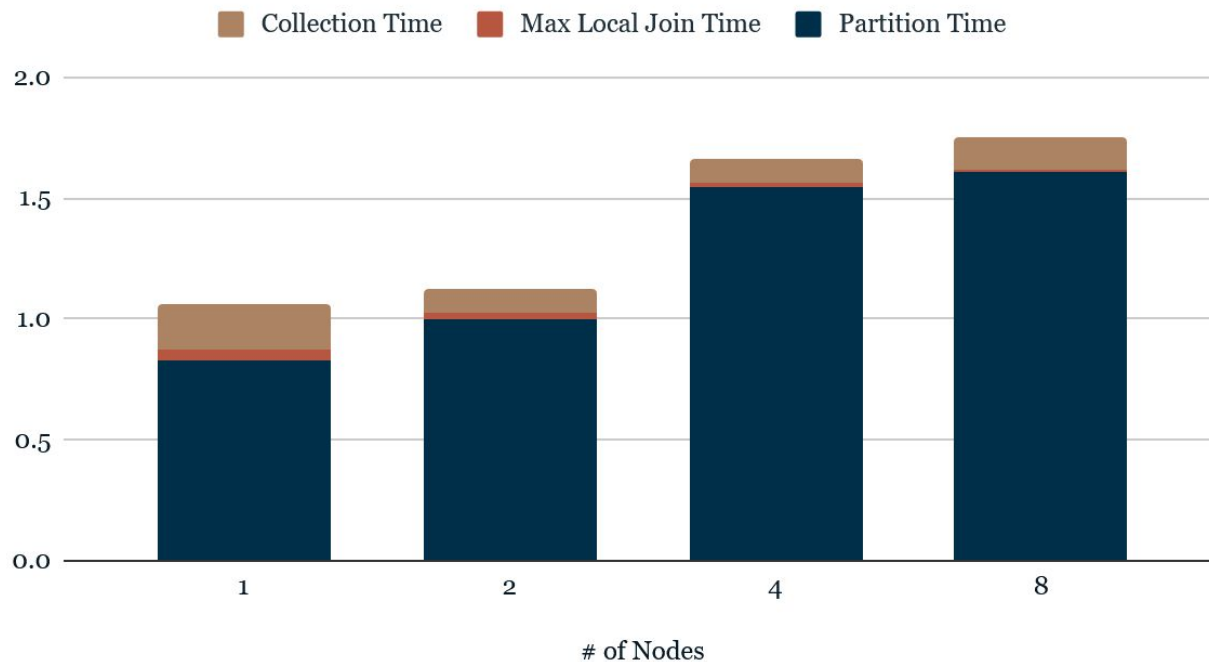
Results (Improved Collection)

Taking the average over 100 jobs. All times are in seconds.

# of Nodes	Total Time	Partition Time	Max Local Join Time	Collection Time
1	1.065717	0.829351	0.043081	0.185991
2	1.129395	0.999377	0.025715	0.09889
4	1.694072	1.549526	0.018069	0.097622
8	1.754642	1.606658	0.011272	0.130829

Results (Improved Collection)

Query 1 Join Time: Streaming Parquet Collection



Partition Code

The coordinator node:

1. Creates a Parquet file for each node containing the sub-table.
2. Sends the Parquet files to the correct node using MPI send.

Each worker node:

1. Receives the Parquet file from node 0 (coordinator).
2. Writes to a local Parquet file.
3. Adds the tables from Parquet to DuckDB.

```
def partition_and_distribute_customer_and_orders_parquet(comm, rank, size):

    db_path = 'data/coordinator/full_data/whole_tpch_0.1.duckdb'
    local_db_path = f'/tmp/node_{rank}.duckdb'
    local_parquet_path_c = '/tmp/customer_local.parquet'
    local_parquet_path_o = '/tmp/orders_local.parquet'

    if rank == 0:

        con = duckdb.connect(db_path, read_only=True)
        for i in range(size):
            part_path_c = f'/tmp/customer_part_{i}.parquet'
            part_path_o = f'/tmp/orders_part_{i}.parquet'

            con.execute(f"""COPY (SELECT * FROM customer WHERE c_custkey % {size} = {i})
                        TO '{part_path_c}' (FORMAT 'parquet')""")
            con.execute(f"""COPY (SELECT * FROM orders WHERE o_custkey % {size} = {i})
                        TO '{part_path_o}' (FORMAT 'parquet')""")

            if i != 0: # coordinator doesn't need to send to itself
                # send both Parquet files
                with open(part_path_c, 'rb') as f:
                    comm.send(f.read(), dest=i, tag=100)
                with open(part_path_o, 'rb') as f:
                    comm.send(f.read(), dest=i, tag=101)
            con.close()

        else:
            file_bytes_c = comm.recv(source=0, tag=100)
            file_bytes_o = comm.recv(source=0, tag=101)

            with open(local_parquet_path_c, 'wb') as f:
                f.write(file_bytes_c)
            with open(local_parquet_path_o, 'wb') as f:
                f.write(file_bytes_o)

    if rank == 0:
        # coordinator reads its own local partitions (you don't send and recv to yourself)
        local_parquet_path_c = f'/tmp/customer_part_{rank}.parquet'
        local_parquet_path_o = f'/tmp/orders_part_{rank}.parquet'

    con = duckdb.connect(local_db_path)
    con.execute("CREATE TABLE IF NOT EXISTS customer AS SELECT * FROM read_parquet(?)", (local_parquet_path_c,))
    con.execute("CREATE TABLE IF NOT EXISTS orders AS SELECT * FROM read_parquet(?)", (local_parquet_path_o,))

    return con
```

Why is this partitioning slow?

Step	Description	Why is it slow?
C2.	Sends the Parquet files to the correct node using MPI send.	Since send is blocking, coordinator can only proceed to sending to the next worker node once the current one receives the message.
W1.	Receives the Parquet file from node 0 (coordinator).	Above issue.

Coordinator Step 2.

```
if i != 0: # coordinator doesn't need to send to itself
    # send both Parquet files
    with open(part_path_c, 'rb') as f:
        comm.send(f.read(), dest=i, tag=100)
    with open(part_path_o, 'rb') as f:
        comm.send(f.read(), dest=i, tag=101)
```

Outline

- Intro to Quest cluster and MPI
- Initial implementation
- **Optimizations**
 - Collection phase
 - **Partitioning phase**
- Takeaways

How can we improve the partitioning phase?

Step	Description	Why is it slow?
C2.	Sends the Parquet files to the correct node using <code>MPI send</code> .	Since <code>send</code> is blocking, coordinator can only proceed to sending to the next worker node once the current one receives the message.
W1.	Receives the Parquet file from node 0 (coordinator).	Above issue.

Use `isend` and `irecv` instead.

Allows coordinator to send orders file to immediately after sending customers.

Partitioning Code

The coordinator node:

1. Exports the partitions of the table to Parquet.
2. Sends Parquet file bytes. To ensure worker node buffer has space:
 - a. Send size of file bytes.
 - b. Send file bytes to be stored in a buffer of that size.

```
if rank == 0:
    con = duckdb.connect(db_path, read_only=True)
    pending_reqs = []

    for i in range(size):
        part_path_c = f'/tmp/customer_part_{i}.parquet'
        part_path_o = f'/tmp/orders_part_{i}.parquet'

        con.execute(f"""
            COPY (
                SELECT * FROM customer WHERE c_custkey % {size} = {i}
            ) TO '{part_path_c}' (FORMAT 'parquet')
        """)

        con.execute(f"""
            COPY (
                SELECT * FROM orders WHERE o_custkey % {size} = {i}
            ) TO '{part_path_o}' (FORMAT 'parquet')
        """)

        with open(part_path_c, 'rb') as f:
            file_bytes_c = f.read()
        with open(part_path_o, 'rb') as f:
            file_bytes_o = f.read()

        if i != 0:
            # issue we had "MPI_ERR_TRUNCATED" (means not enough space in buffer)
            # to resolve: manually allocate recvbuffer size using Isend instead of isend
            # (isend sends a Python object, Isend allows for lower level)
            # send sizes
            size_c = len(file_bytes_c)
            size_o = len(file_bytes_o)
            req_size_c = comm.isend(size_c, dest=i, tag=200)
            req_size_o = comm.isend(size_o, dest=i, tag=201)

            # then send actual data
            req_data_c = comm.Isend([file_bytes_c, MPI.BYTE], dest=i, tag=100)
            req_data_o = comm.Isend([file_bytes_o, MPI.BYTE], dest=i, tag=101)

            pending_reqs.extend([req_size_c, req_size_o, req_data_c, req_data_o])

        # clean up local partition files
        if i != 0:
            os.remove(part_path_c)
            os.remove(part_path_o)

    MPI.Request.Waitall(pending_reqs)
    con.close()
```

Partitioning Code

Each worker node:

1. Receives the size of file bytes and allocates space.
2. Receives the data (before doing anything with it, must call Wait() to ensure no further changes).
3. Write file to Parquet and make table in local DuckDB.

```
else:
    # receive sizes
    req_size_c = comm.irecv(source=0, tag=200)
    req_size_o = comm.irecv(source=0, tag=201)
    size_c = req_size_c.wait()
    size_o = req_size_o.wait()

    # alloc buffers
    file_bytes_c = bytearray(size_c)
    file_bytes_o = bytearray(size_o)

    # receive actual data
    req_data_c = comm.Irecv([file_bytes_c, MPI.BYTE], source=0, tag=100)
    req_data_o = comm.Irecv([file_bytes_o, MPI.BYTE], source=0, tag=101)

    # must wait until message is completed
    req_data_c.Wait()
    req_data_o.Wait()

    with open(local_parquet_path_c, 'wb') as f:
        f.write(file_bytes_c)
    with open(local_parquet_path_o, 'wb') as f:
        f.write(file_bytes_o)

    # Load into local DuckDB
    if rank == 0:
        local_parquet_path_c = f'/tmp/customer_part_{rank}.parquet'
        local_parquet_path_o = f'/tmp/orders_part_{rank}.parquet'

    con = duckdb.connect(local_db_path)
    con.execute("DROP TABLE IF EXISTS customer")
    con.execute("DROP TABLE IF EXISTS orders")
    con.execute("CREATE TABLE customer AS SELECT * FROM read_parquet(?)", (local_parquet_path_c,))
    con.execute("CREATE TABLE orders AS SELECT * FROM read_parquet(?)", (local_parquet_path_o,))

    return con
```

Results (Improved Partitioning)

Taking the average over 100 jobs. All times are in seconds.

# of Nodes	Total Time	Partition Time	Max Local Join Time	Collection Time
1	0.863646	0.696623	0.039775	0.121561
2	1.043517	0.82995	0.023754	0.187868
4	0.908775	0.726448	0.016083	0.145061
8	0.778797	0.61511	0.010027	0.150962

Results (Improved Partitioning)

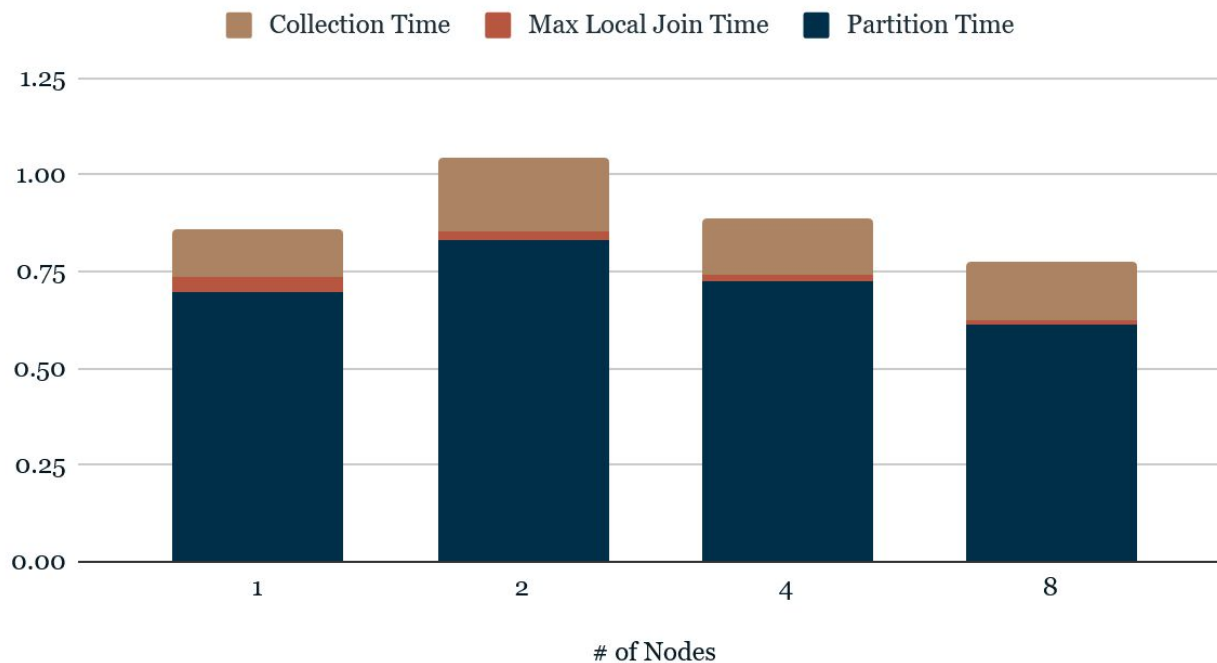
# of Nodes	Total Time	Partition Time	Max Local Time	Collection Time
1	0.863646	0.696623	0.039775	0.121561
2	1.043517	0.82995	0.023754	0.187868
4	0.908775	0.726448	0.016083	0.145061
8	0.778797	0.61511	0.010027	0.150962

Previous

# of Nodes	Total Time	Partition Time	Max Local Time	Collection Time
1	1.065717	0.829351	0.043081	0.185991
2	1.129395	0.999377	0.025715	0.09889
4	1.694072	1.549526	0.018069	0.097622
8	1.754642	1.606658	0.011272	0.130829

Results (Improved Partitioning)

Query 1 Join Time: Streaming Parquet Partitioning



Outline

- Intro to Quest cluster and MPI
- Initial implementation
- Optimizations
 - Collection phase
 - Partitioning phase
- **Takeaways**

Takeaways

- The biggest bottleneck is partitioning and collecting across nodes (i.e., communication overhead).
- Use non-blocking operations for overlapping communication.

Questions and Next Steps

- How do these perform with extremely large datasets, i.e., at what point, if any, does this collecting scheme yield benefits as nodes are added? → *Use TPC-H with different scale.*
- Why does performance time decrease as nodes are added in the implementation with improved partitioning? Why does the collection time increase?
- The standard deviation of the measured times is not great. Why do the jobs differ so much?
- How can we parallelize the process more (e.g. on each node, run multiple processes with Python multiprocessing)?

...

Thank you!